



FACULTY OF INFORMATION TECHNOLOGY

ASSIGNMENT 1 - INDIVIDUAL (20%)

COURSE TITLE: OBJECT ORIENTED PROGRAMMING

COURSE CODE: BIT1123

ASSIGNMENT TITLE: JAVA LAB WORKOUT

PREPARED BY: MD BAHAUDDIN

ID: 202402010057

PASSPORT: A08365770

GitHub Repository URL: https://github.com/bahauddinpalash/Java_tutorial

1. Executive Summary

This document summarizes the practical implementations completed during the weekly laboratory sessions for the BIT1123 Object-Oriented Programming course. Over the 10-week curriculum, fundamental Java programming concepts, object-oriented design paradigms, exception management, collections, and file I/O operations were developed, executed, and maintained using GitHub Codespaces.

2. Weekly Tasks Summary Table

Week	Topic Covered	Key Concepts Implemented	Source Files
Week 1	Introduction & Environment Setup	Core Java syntax, standard console output streams	week 1/HelloWorld.java
Week 2	Variables & Data Types	Primitive data structures, string formatting, memory handling	week 2/StudentWeek1.java
Week 3	Control Structures	Conditional branching (if-else), iterative loops (for, while)	week 3/ControlStructures.java
Week 4	Methods & Functions	Method declarations, parameter	week 4/MethodsDemo.java

Week	Topic Covered	Key Concepts Implemented	Source Files
		passing, return values	
Week 5	Classes & Objects	Encapsulation principles, class constructors, instance attributes	week 5/Student.java, week 5/Main.java
Week 6	Inheritance	Parent-child class hierarchies, extends keyword, method overriding	week 6/Employee.java, week 6/Lecturer.java, week 6/Main.java
Week 7	Abstraction	Abstract class structure, interface contract implementations	week 7/Appliance.java, week 7/WashingMachine.java, week 7/Main.java
Week 8	Exception Handling	try-catch exception blocks, error handling resilience, finally blocks	week 8/ExceptionHandlingDemo.java
Week 9	Java Collections Framework	Dynamic array management (ArrayList),	week 9/CollectionsDemo.java

Week	Topic Covered	Key Concepts Implemented	Source Files
		generics, collection iteration	
Week 10	File I/O Operations	Data writing streams (FileWriter), error catching, persistent file storage	week 10/FileIODemo.java

3. Key Modules and Implementations

3.1 Basics (Weeks 1–4)

During the first stretch, we mainly tried to build a solid feel for core Java syntax. A bunch of small programs were made to handle different data structures, steer the run sequence through conditional logic, and set up reusable methods, so the code stays tidier, and not just scattered.

3.2 Object–Oriented Foundations (Weeks 5–7)

- Encapsulation (Week 5): Wrapped the Student attributes inside the student class with private modifiers, so the information stays consistent, by limiting access using purpose-built getter and setter methods.
- Inheritance (Week 6): Created the Lecturer subclass from the Employee parent, using the extends keyword, this also cut down duplicate code for properties that both classes share.

- Abstraction (Week 7): Defined abstract rules in the Appliance base class, forcing derived components like WashingMachine to actually follow those contracts, more or less.

3.3 Advanced Ideas (Weeks 8–10)

- Exception Handling (Week 8): Dealt with runtime issues (for example division by zero) using formal try-catch blocks, so the whole app doesn't just crash out without warning.
- Collections (Week 9): Worked with the ArrayList type, to handle changing datasets without sticking to fixed-size array limits.
- File Operations (Week 10): Used FileWriter output streams to stash log entries straight into outside text files, pretty direct.

4. GitHub Repository Status

All source files have been compiled, executed, and pushed to the remote GitHub repository. Project history and source code versions were managed using structured Git commits. The repository is publicly hosted at:

https://github.com/bahauddinpalash/Java_tutorial

5. Conclusion

The practical lab workouts successfully satisfy all requirements outlined in the BIT1123 course module. All Java components function properly without runtime errors, adhering to core object-oriented principles and established programming standards.